

HACKATHON · THEME 2 · TROUBLESHOOTING

Smart Guided Troubleshooting Engine Engine

From a vague phone complaint to a verified, ordered fix plan where every Settings step is one tap away.

AUTO

MANUAL

CRITICAL

CodeSwitch - Rayasam Amarnath · Neeraj Prasad

01 • THE PROBLEM

Every complaint costs an agent 15 minutes

15 min

of manual triage per scenario, across millions of customer interactions.

"My Nexa X1 screen inputs are delayed and the touch responsiveness is laggy."

1. An agent reads long knowledge-base articles to find the cause.
2. The agent picks and orders the right steps by hand.
3. The customer hunts through nested Settings menus alone.

02 · WHAT WE BUILT

One request in, a verified plan out

INPUT · CUSTOMER WORDS

"My Nexa X1 screen inputs are delayed and the touch responsiveness is laggy, causing a noticeable delay."

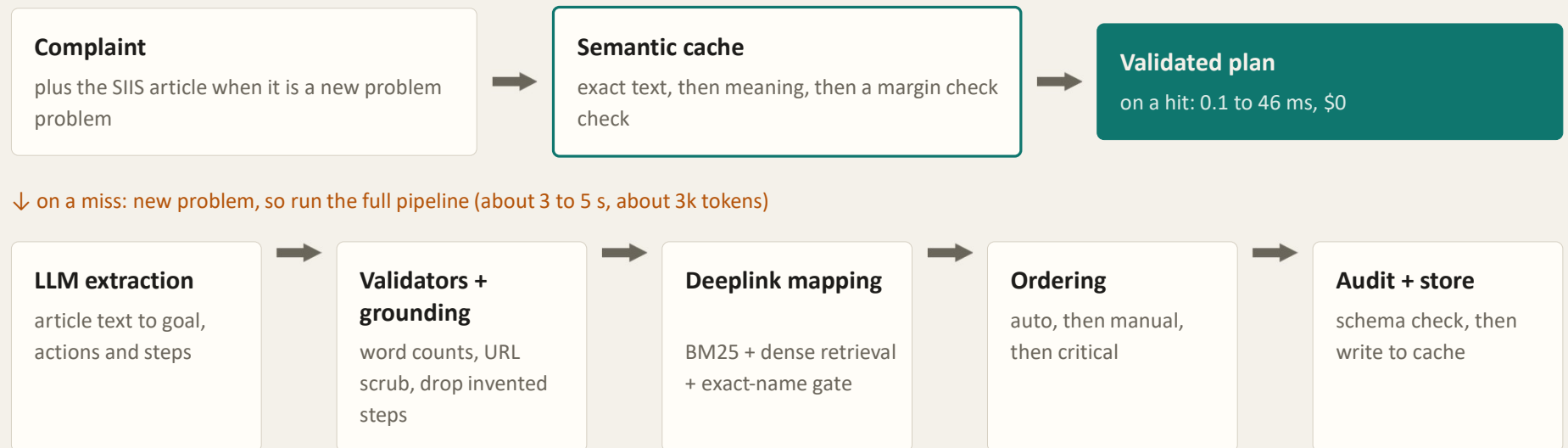


OUTPUT · 8 ACTIONS, SAFEST FIRST

01 Enable Touch Sensitivity	AUTO
02 Disable Full Screen Gestures	AUTO
03 Turn Off Touch Sensitivity	AUTO
04 Remove Screen Protector	MANUAL
05 Swap Charger	MANUAL
06 Restart Device	CRITICAL
07 Enter Safe Mode	CRITICAL
08 Factory Data Reset	CRITICAL

03 · ARCHITECTURE

Cache first, full pipeline only on a miss



The only LLM call is extraction. Everything after it is deterministic code.

04 · DESIGN PRINCIPLE

The LLM reads. Code decides.

The model only restructures article text. Rules, deeplinks, categories and ordering are enforced in code, so a deeplink can never be invented.

Four checks on every answer

✓ Rules in code

Word counts, Title Case and goal syntax are validated and auto-auto-fixed after every LLM call.

🔒 Zero URL leaks

Links, domains and emails are scrubbed, including an email email hidden inside one source article.

🔍 Grounding check

Every step must trace back to the article. An unstated “Release the buttons.” was caught and dropped.

🛡️ Schema + pure JSON

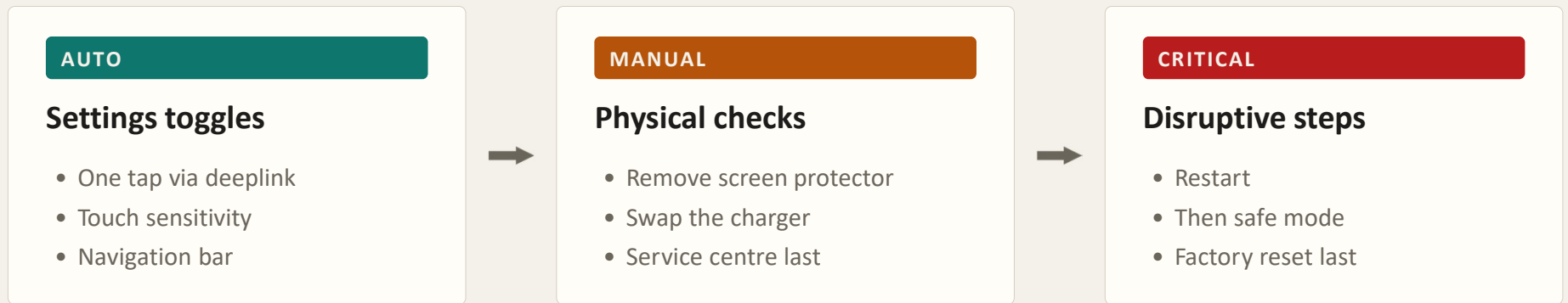
Every response is re-validated against schema.py before it leaves the API.

Exact screen, never a look-alike

Retrieval proposes candidates from 550 phone deeplinks. A word-order-aware name gate accepts one only if its setting name matches a screen the step actually names.

Screen named in the step	Closest catalog entry	Result
Super steady mode	Easy mode	Rejected
Navigation bar	Hide status and navigation bars	Rejected
Factory data reset	Auto factory reset	Rejected
Screen lock and biometrics	Lock screen	Rejected
Touch sensitivity, on / off	Enable / Disable Touch sensitivity	Matched by verb
Back up data	Back up data (TechCorp Cloud)	Same link as official sample

Least disruptive first, destructive last



A stable sort keeps the article's own order inside each group, and the auditor flags any plan that breaks the rule.

A cache that refuses to guess

Each validated plan is stored with about 10 phrasings: 17 phrasings: 17 plans, 174 ways in. A match must be close close **and** clearly beat every other plan.

86%

hit rate on unseen, hand-written paraphrases

0

wrong-plan hits and false hits

39 ms

P95 for a paraphrase, 0.1 ms for an exact repeat

Threshold	Hit rate	False hits
0.800	90%	2
0.825	88%	1
0.850 (chosen)	86%	0
0.875	72%	0
0.900	65%	0

We chose the lowest threshold with zero mistakes of any kind. Near-ties fall back to the full pipeline instead of guessing.

Measured against the problem statement's targets

Metric	Target	Ours
Schema-valid outputs	$\geq 99\%$	100% (20/20)
Absolute URL leaks	0	0
Deeplinks found in the catalog	100%	100%
Auto actions with a deeplink	$\geq 90\%$	100%
Cache hit P95, exact repeat (HTTP)	≤ 300 ms	5.6 ms
Cache hit P95, unseen paraphrase (HTTP)	≤ 300 ms	45.9 ms
Unseen paraphrase hit rate	$\geq 80\%$	86%
Cold path P95, full pipeline	≤ 8 s	[__ s]
Problems in 160 concurrent requests	0	0

Why the name gate matters

5 of 7

auto actions where plain top-1 retrieval chose
chose a screen the steps never name.

Variant	Outcome
Full LLM deeplink mapping	Not used: masked URIs invite invented links
Hybrid retrieval, top-1 only	Picks decoys such as Easy mode for Super steady mode
Hybrid retrieval + name gate (ours)	Only exact screens; a labelled placeholder otherwise
Pure rules, offline	Free and deterministic, but coarse actions

11 • FINDINGS

What the data taught us

Samples break the rules

9- and 12-word descriptions; bixby:// vs voiceassist://.

Mismatched articles

3 complaints got unrelated articles: honest no_match.

LLM output varies

Even at temperature 0; the cache makes repeats identical.

Catalog decoys

Look-alike names and misleading messages; we match the key.

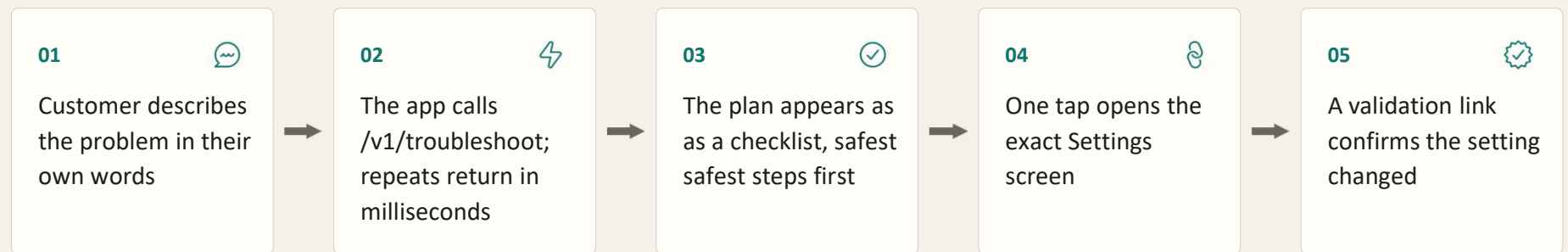
Non-phone entries

TV, fridge and air-conditioner links filtered out.

Free-tier limits

Cold runs are paced; failed calls are never timed.

Behind the support app, on every phone



Agents are freed from routine cases, customers get consistent help at any hour, and cost stays near zero because repeats never reach the LLM.

What we would build next



Full RAG over SIIS

Retrieve the right article automatically, which also fixes mismatched pairs.



Vector index at scale

Move to FAISS or a managed vector store once phrasings reach the hundreds of thousands.



Multi-intent complaints

Split “cracked and touch not working” into separate goals.



On-device client

An Android client that renders plans and fires deeplinks natively.

THANK YOU

Questions?

Live demo console, REST API with /docs, 100 automated tests, and a full metrics report in the repository.

[github.com/Rayasam3/smart-troubleshooting-engine]

CodeSwitch - Rayasam Amarnath · Neeraj Prasad
Prasad